GPS Tracking System | Testing Report

1. Testing Overview and Strategy

The GPS Tracking System was tested using a combination of automated JUnit testing and manual GUI verification to evaluate its functionality, reliability, and compliance with the assignment requirements

Automated tests were developed to verify individual components, including distance calculations, coordinate restrictions, five-minute GPS history management, and altitude conversion. Additional tests focused on the Functional Reactive Programming (FRP) implementation, particularly Sodium's event filtering, stream transformations, and reactive state accumulation.

Manual GUI testing was conducted to verify the behaviour of the complete application, including the display of ten GPS trackers, live coordinate updates, dynamic restrictions, input validation, and responsiveness during continuous GPS updates.

This combined approach was intended to identify errors at both the component and application levels, while verifying that the reactive processing logic produced the expected behaviour in the user interface.

2. Test environment and execution

The implementation uses Java 11, Sodium 1.2.0, Swing, and JUnit 4.13.2 with Hamcrest 1.3. The application receives GPS events from the supplied GpsService; that service was not modified during this implementation. Commands below assume Windows PowerShell, the repository root as the working directory, and the required dependencies available locally.

```
$classpath = ".;lib/sodium-1.2.0.jar;<path-to-junit-4.13.2.jar>;<path-to-hamcrest-core-1.3.jar>"

javac -cp $classpath -d . GpsGUI.java GpsEventDisplay.java `
  GpsEvent.java GpsService.java GpsDistance.java `
  GpsRestrictions.java GpsHistory.java

java -cp $classpath GpsGUI
```

Replace the JUnit and Hamcrest placeholders with local absolute paths. Ensure the supplied GPS input data required by GpsService is available at runtime. These local dependencies and supplied GPS files should not be treated as newly authored implementation files.

3. Automated JUnit verification

The following eight test classes were compiled and executed together. The class totals and method names were collected directly from the repository via PowerShell. The final run returned OK (34 tests), with no reported failures.

Class	Tests	Areas verified
GpsDistance_Test	8	3D movement, altitude conversion, symmetry, zero movement
GpsDistanceWindow_Test	9	5-minute window, boundary, reset, rounding, restrictions
GpsRestrictions_Test	10	Accepted/rejected coordinates, bounds, zero and full ranges
GpsAltitudeFRP_Test	2	Sodium map() altitude conversion and metre-valued distance
GpsEventDisplay_Test	2	Three-second clearing and timer restart
GpsFiltering_Test	1	Sodium filtering
GpsHistory_Test	1	History and accumulated distance
GpsHistoryFRP_Test	1	Sodium accum() state management
TOTAL	34	All passed in one final JUnit run

Reproduce the automated suite

```
$tests = @("GpsDistance_Test", "GpsDistanceWindow_Test", `
 "GpsRestrictions_Test", "GpsAltitudeFRP_Test", `
 "GpsEventDisplay_Test", "GpsFiltering_Test", `
 "GpsHistory_Test", "GpsHistoryFRP_Test")

# Compile the eight test sources and their dependencies
javac -cp $classpath -d . GpsEvent.java GpsDistance.java `
 GpsRestrictions.java GpsDistanceWindow.java GpsHistory.java `
 GpsEventDisplay.java ($tests | ForEach-Object { "$_.java" })
```

```
PS C:\Users\isobe\OneDrive\Desktop\2026\Event Driven Computing\Assignment 3\EDC-Assignm
ent-3> java -cp $classpath org.junit.runner.JUnitCore GpsDistance_Test GpsDistanceWindo
w_Test GpsRestrictions_Test GpsAltitudeFRP_Test GpsEventDisplay_Test GpsFiltering_Test
GpsHistory_Test GpsHistoryFRP_Test
JUnit version 4.13.2
..................................
Time: 8.335

OK (34 tests)

PS C:\Users\isobe\OneDrive\Desktop\2026\Event Driven Computing\Assignment 3\EDC-Assignm
ent-3>
```

Figure 1: Successful execution of all 34 JUnit tests across eight test classes, with no failures.

4. Automated test inventory

GpsDistance_Test

testNoMovement; testFeetToMetres; testAltitudeOnlyMovement; testLatitudeMovement; testLongitudeMovement; testDistanceSymmetry; testCombinedMovement; testDescendingAltitude.

GpsDistanceWindow_Test

testInitialDistance; testMultipleMovements; testFiveMinuteExpiry; testRoundedDistance; testReset; testExactFiveMinuteBoundary; testLongGapBetweenEvents; testDistanceRecalculationAfterRestrictionChange; testDistanceWithAltitudeAlreadyInMetres.

GpsRestrictions_Test

testEventInsideRange; testLatitudeOutsideRange; testLongitudeOutsideRange; testBothCoordinatesOutsideRange; testPositiveBoundaries; testNegativeBoundaries; testZeroRestrictionAcceptsOrigin; testZeroRestrictionRejectsMovement; testAltitudeDoesNotAffectFiltering; testFullCoordinateRange.

GpsAltitudeFRP_Test

testAltitudeConversionUsingMap; testDistanceWithAltitudeAlreadyInMetres.

GpsEventDisplay_Test

testEventClearsAfterThreeSeconds; testNewEventRestartsTimer.

GpsFiltering_Test

testSodiumFiltering.

GpsHistory_Test

testHistoryAccumulatesDistance.

GpsHistoryFRP_Test

testReactiveHistoryAccumulation.

5. Manual GUI tests and observed results

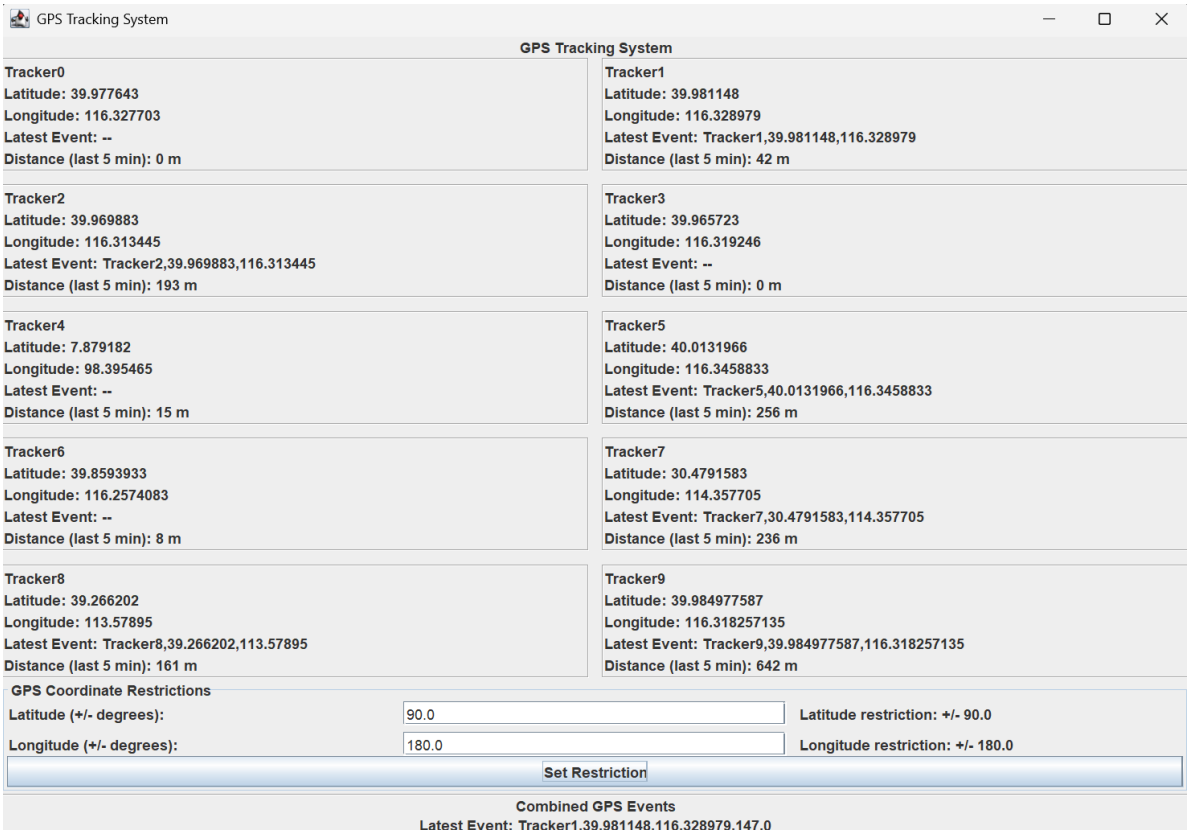


Figure 2: GPS tracking GUI displaying ten trackers, live coordinates, five-minute distance calculations, and the combined event stream with unrestricted coordinates.

Test Scenario	Test Input / Action	Expected Result	Actual Result	Status
Normal operation	Launch GUI with all 10 trackers	All trackers display live coordinates and distances	All 10 trackers displayed; coordinates and distances updated	PASS
Combined restrictions	Set latitude and longitude to 0, then restore 90 and 180	Distances become 0 m, then recalculate using retained history	All distances became 0 m; previously accumulated distances reappeared	PASS
Latitude-only filtering	Latitude = 40, longitude = 180	Positions outside ±40° latitude are excluded	Tracker5 exceeded 40° and displayed 0 m; eligible trackers continued updating	PASS
Longitude-only filtering	Latitude = 90, longitude = 115	Positions outside ±115° longitude are excluded	Trackers near 116° displayed 0 m; eligible trackers continued accumulating distance	PASS
Invalid latitude	Latitude = 100	Display error without changing active restrictions	Error displayed; previous restrictions preserved	PASS
Invalid longitude	Longitude = 200	Display error without changing active restrictions	Error displayed; previous restrictions preserved	PASS
Empty input	Leave longitude empty	Reject missing value	Correct validation error displayed	PASS
Non-numeric input	Enter abc for longitude	Reject non-numeric value	Correct validation error displayed	PASS
Non-finite input	Enter NaN for longitude	Reject non-finite number	Correct validation error displayed	PASS
Event clearing	Send event, then allow 3 seconds without updates	Event disappears after 3 seconds	Event cleared after approximately 3 seconds in controlled JUnit testing	PASS*
Timer reset	Send another event before timer expires	Three-second countdown restarts	Second event restarted the timer in JUnit testing	PASS*
GUI responsiveness	Interact with GUI during GPS updates	GUI remains responsive	No noticeable freezing during manual interaction	MANUAL PASS

*PASS** indicates that the behaviour was verified through controlled JUnit testing rather than an end-to-end GUI test. GUI responsiveness was assessed manually and was not subject to formal performance benchmarking.

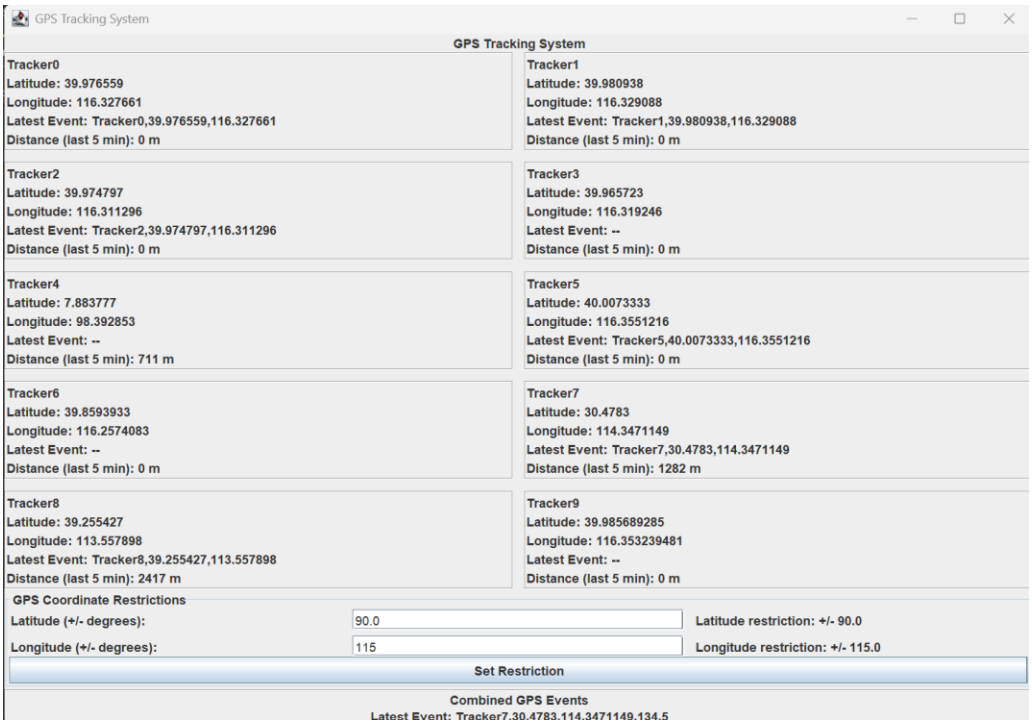


Figure 3: Longitude-only restriction testing (latitude ±90°, longitude ±115°), demonstrating that out-of-range trackers contribute no distance while eligible trackers continue accumulating movement.

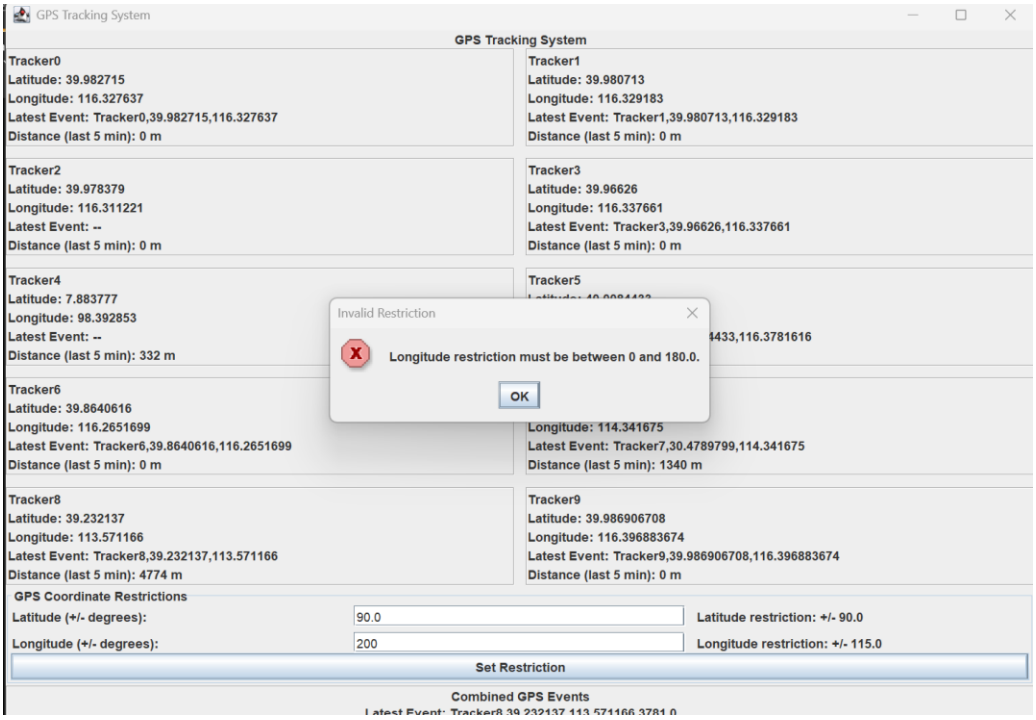


Figure 4: Invalid longitude input (200°) produces a validation error while preserving the previously accepted coordinate restrictions.

6. FRP Implementation Checkpoints

The GPS Tracking System uses Sodium's Functional Reactive Programming (FRP) primitives to process incoming GPS events, maintain tracker history, apply coordinate restrictions, and update the graphical interface. Testing focused on verifying both the individual transformations and their integration into the application.

FRP Primitive	Implementation and Purpose	Testing and Verification
map()	Transforms GPS events by removing altitude from simplified tracker displays, converting altitude from feet to metres, and attaching timestamps to incoming events.	GpsAltitudeFRP_Test verifies altitude conversion using map(). GUI testing confirms simplified tracker displays exclude altitude.
orElse()	Combines the ten individual GPS event streams into a single stream for the combined GPS display.	Manual GUI testing confirms that the combined display receives events from multiple trackers.
accum()	Maintains each tracker's GPS history as an immutable reactive state, supporting distance calculations over the previous five minutes.	GpsHistoryFRP_Test verifies that incoming events update the history maintained by Sodium.
lift()	Combines GPS history with the active coordinate restrictions, recalculating distance when either changes.	Manual restriction testing confirms that displayed distances respond to changes without discarding GPS history.
snapshot()	Applies the current coordinate restrictions to combined GPS events and recalculates distances using the current time during periodic refreshes.	GpsFiltering_Test verifies Sodium filtering. Manual GUI testing confirms coordinate restrictions and distance updates.
listen()	Connects reactive outputs to the graphical interface, scheduling updates to Swing components through the Event Dispatch Thread.	Manual GUI testing confirms that GPS coordinates, recent events, and distance labels update during operation.

Together, these operations separate GPS event processing from the presentation layer. Sodium manages the reactive data transformations and history updates, while Swing displays the resulting information. The automated tests and manual GUI verification provide evidence that the implemented FRP operations support the required tracking and filtering functionality.

7.1 Testing Limitations

Although all 34 automated JUnit tests passed, several limitations should be acknowledged. First, high-frequency GPS event processing has not been formally stress-tested. While the GUI remained responsive during normal operation, this does not establish its performance under significantly increased event rates. Second, automated testing focused primarily on individual components and selected FRP operations. Although manual GUI testing confirmed that coordinate restrictions affected displayed distances, the complete reactive processing pipeline was not independently tested under every possible event sequence or timing condition. Third, the three-second event-clearing functionality was verified through controlled JUnit tests of GpsEventDisplay. However, its timing was not formally measured as part of an end-to-end GUI test with the supplied GPS service. Finally, distance calculations were verified using defined test cases, including altitude changes, coordinate movement, rounding, and five-minute expiry. However, the application was not compared against an independent GPS tracking system or external reference dataset.

7.2 Testing Conclusion

The GPS Tracking System was evaluated using 34 automated JUnit tests across eight test classes, supported by manual GUI testing. All 34 tests passed without reported failures. The testing covered GPS distance calculations, coordinate filtering, altitude conversion, reactive history accumulation, event clearing, and user input validation. Manual testing also confirmed that the application displayed ten GPS trackers, processed continuous GPS updates, and responded to changes in coordinate restrictions. Overall, the results provide evidence that the implemented system satisfies the principal functional requirements examined during testing. While additional stress testing and broader end-to-end verification could further strengthen confidence, the completed tests demonstrate consistent behaviour across the assessed scenarios.